HPC (lab_02)

Exercise 1: Synchronization Mechanisms in OpenMP

Objective: Write an OpenMP program to demonstrate various synchronization mechanisms. Your implementation should cover the following five approaches:

1. **Implicit Barriers:** Demonstrating how work-sharing constructs (like #pragma omp parallel) have a built-in barrier at the end. In fork – join block join block act as barrier.
2. **Implicit Barriers with Threadprivate Variables:** Using threadprivate data to show how persistence a particular variable retains its value across parallel regions interacts with synchronization.
3. **Explicit Barriers:** Implementing the #pragma omp barrier directive to manually synchronize all threads in a team.
4. **The single Directive:** Using #pragma omp single to ensure a block of code is executed by only one thread, with an implicit barrier for others. Also use this directive with nowait clause to see the execution of program.
5. **The master Directive:** Using #pragma omp master to execute code only on the master thread (noting the lack of an implicit barrier compared to single).

```c
#include <omp.h>
#include <stdio.h>

int numt;

int main()
{
    #pragma omp parallel
    {
        int tid = omp_get_thread_num();

        # pragma omp single nowait
        numt = omp_get_num_threads();

        printf("Hello World from Thread %d of %d\n", tid, numt);
    }
}
```

```
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB$ cd 02_lab_
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/02_lab_$ ./p1
Hello World from Thread 0 of 16
Hello World from Thread 15 of 16
Hello World from Thread 13 of 16
Hello World from Thread 14 of 0
Hello World from Thread 1 of 0
Hello World from Thread 4 of 0
Hello World from Thread 12 of 0
Hello World from Thread 3 of 0
Hello World from Thread 7 of 0
Hello World from Thread 6 of 0
Hello World from Thread 9 of 0
Hello World from Thread 2 of 0
Hello World from Thread 11 of 16
Hello World from Thread 10 of 16
Hello World from Thread 8 of 16
Hello World from Thread 5 of 16
```

Observations:

```
All threads execute concurrently inside the parallel region.

The statement after the parallel block executes only after all threads
complete.

This demonstrates the fork-join model, where join acts as an implicit barrier.

Execution order of thread prints is non-deterministic.
```

02_lab_ > C 02_lab2_.c

```c

int counter = 0;
#pragma omp threadprivate(counter)

int main() {

    #pragma omp parallel
    {
        counter = omp_get_thread_num();
        printf("Thread %d counter value: %d\n",
               omp_get_thread_num(), counter);
    } // implicit barrier

    #pragma omp parallel
    {
        printf("Thread %d retained counter value: %d\n",
               omp_get_thread_num(), counter);
    }

    return 0;
}

/*
-->*Observation s:

- Each thread has its own copy of the variable.

- Values written by threads are remembered for later use.

- The program shows how OpenMP keeps thread data safe.

- Implicit barrier ensures correct and ordered execution.


*/

```

```
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/02_lab_$ ./p2
Thread 2 counter value: 2
Thread 11 counter value: 11
Thread 3 counter value: 3
Thread 7 counter value: 7
Thread 15 counter value: 15
Thread 6 counter value: 6
Thread 8 counter value: 8
Thread 4 counter value: 4
Thread 5 counter value: 5
Thread 12 counter value: 12
Thread 10 counter value: 10
Thread 1 counter value: 1
Thread 9 counter value: 9
Thread 14 counter value: 14
Thread 13 counter value: 13
Thread 0 counter value: 0
Thread 0 retained counter value: 0
Thread 1 retained counter value: 1
Thread 7 retained counter value: 7
Thread 6 retained counter value: 6
Thread 9 retained counter value: 9
Thread 4 retained counter value: 4
Thread 13 retained counter value: 13
Thread 8 retained counter value: 8
Thread 12 retained counter value: 12
Thread 15 retained counter value: 15
Thread 10 retained counter value: 10
Thread 14 retained counter value: 14
Thread 3 retained counter value: 3
Thread 11 retained counter value: 11
Thread 5 retained counter value: 5
Thread 2 retained counter value: 2
```

02_lab_ > C 03_lab_.c

```c
#include <stdio.h>
#include <omp.h>

int main() {
    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        printf("Thread %d reached point A\n", tid);

        #pragma omp barrier

        printf("Thread %d passed the barrier\n", tid);
    }
    return 0;
}

OBservations:


- Multiple threads run in parallel and print a message when they reach point A.

- Each thread then waits at the barrier until all other threads arrive.

- The barrier ensures that no thread moves ahead before synchronization.

- After synchronization, all threads continue and print the second message.
```

```
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/02_lab_$ ./p3
Thread 0 reached point A
Thread 11 reached point A
Thread 10 reached point A
Thread 6 reached point A
Thread 4 reached point A
Thread 15 reached point A
Thread 5 reached point A
Thread 1 reached point A
Thread 8 reached point A
Thread 3 reached point A
Thread 12 reached point A
Thread 14 reached point A
Thread 2 reached point A
Thread 7 reached point A
Thread 13 reached point A
Thread 9 reached point A
Thread 11 passed the barrier
Thread 15 passed the barrier
Thread 13 passed the barrier
Thread 7 passed the barrier
Thread 10 passed the barrier
Thread 6 passed the barrier
Thread 14 passed the barrier
Thread 9 passed the barrier
Thread 0 passed the barrier
Thread 5 passed the barrier
Thread 3 passed the barrier
Thread 2 passed the barrier
Thread 4 passed the barrier
Thread 8 passed the barrier
Thread 12 passed the barrier
Thread 1 passed the barrier
```

02_lab_ > C 04a_lab2_.c

```c
#include <stdio.h>
#include <omp.h>

int main() {
    #pragma omp parallel
    {
        printf("Thread %d before single\n", omp_get_thread_num());

        #pragma omp single
        {
            printf("Single executed by thread %d\n",
                   omp_get_thread_num());
        }

        printf("Thread %d after single\n", omp_get_thread_num());
    }
    return 0;
}

// Observations:
- Only one thread executes the single block, while others wait.
- The thread that executes the single block is non-deterministic.
- After the single block, all threads continue execution.
- The implicit barrier at the end of the single block ensures
  synchronization before proceeding.
- Execution order of threads before and after the single block is non-deterministic.
- The program demonstrates the use of the single directive in OpenMP for executing
  a block of code by only one thread while others wait.
```

```
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/02_lab_$ gcc -fopenmp 04a_lab2_.c  -o p4a
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/02_lab_$ ./p4a
Thread 4 before single
Single executed by thread 4
Thread 13 before single
Thread 15 before single
Thread 3 before single
Thread 5 before single
Thread 7 before single
Thread 0 before single
Thread 1 before single
Thread 8 before single
Thread 6 before single
Thread 2 before single
Thread 12 before single
Thread 14 before single
Thread 11 before single
Thread 10 before single
Thread 9 before single
Thread 10 after single
Thread 7 after single
Thread 13 after single
Thread 1 after single
Thread 2 after single
Thread 14 after single
Thread 8 after single
Thread 3 after single
Thread 0 after single
Thread 12 after single
Thread 4 after single
Thread 6 after single
Thread 9 after single
Thread 5 after single
Thread 11 after single
Thread 15 after single
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/02_lab_$
```

02_lab_ > C 04a_lab2_.c

```c
#include <stdio.h>
#include <omp.h>

int main() {
    #pragma omp parallel
    {
        printf("Thread %d before single\n", omp_get_thread_num());

        #pragma omp single
        {
            printf("Single executed by thread %d\n",
                   omp_get_thread_num());
        }

        printf("Thread %d after single\n", omp_get_thread_num());
    }
    return 0;
}

// Observations:
- Only one thread executes the single block, while others wait.
- The thread that executes the single block is non-deterministic.
- After the single block, all threads continue execution.
- The implicit barrier at the end of the single block ensures
  synchronization before proceeding.
- Execution order of threads before and after the single block is non-deterministic.
- The program demonstrates the use of the single directive in OpenMP for executing
  a block of code by only one thread while others wait.
```

```
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/02_lab_$ ./p4b
Thread 15 before single
Single executed by thread 15
Thread 15 after single
Thread 9 before single
Thread 9 after single
Thread 12 before single
Thread 12 after single
Thread 13 before single
Thread 13 after single
Thread 3 before single
Thread 3 after single
Thread 5 before single
Thread 5 after single
Thread 1 before single
Thread 1 after single
Thread 0 before single
Thread 0 after single
Thread 2 before single
Thread 2 after single
Thread 10 before single
Thread 10 after single
Thread 11 before single
Thread 11 after single
Thread 6 before single
Thread 6 after single
Thread 4 before single
Thread 4 after single
Thread 7 before single
Thread 7 after single
Thread 14 before single
Thread 14 after single
```

02_lab_ > C 05_lab2_.c

```c
#include <stdio.h>
#include <omp.h>

int main() {
    #pragma omp parallel
    {
        printf("Thread %d before master\n", omp_get_thread_num());

        #pragma omp master
        {
            printf("Master thread executing this section\n");
        }

        printf("Thread %d after master\n", omp_get_thread_num());
    }
    return 0;
}


// Observations:
- Only the master thread (thread 0) executes the master block.
- Other threads do not wait and continue execution immediately after the master block.
- The execution order of threads before and after the master block is non-deterministic.
- The program demonstrates the use of the master directive in OpenMP for executing
  a block of code exclusively by the master thread without causing other threads to wait.
```

```
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/02_lab_$ gcc -fopenmp 05_lab2_.c  -o p5
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/02_lab_$ ./p5
Thread 5 before master
Thread 5 after master
Thread 1 before master
Thread 1 after master
Thread 4 before master
Thread 4 after master
Thread 11 before master
Thread 11 after master
Thread 2 before master
Thread 2 after master
Thread 15 before master
Thread 15 after master
Thread 14 before master
Thread 14 after master
Thread 9 before master
Thread 9 after master
Thread 8 before master
Thread 8 after master
Thread 12 before master
Thread 12 after master
Thread 6 before master
Thread 6 after master
Thread 0 before master
Master thread executing this section
Thread 0 after master
Thread 3 before master
Thread 3 after master
Thread 10 before master
Thread 10 after master
Thread 13 before master
Thread 13 after master
Thread 7 before master
Thread 7 after master
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/02_lab_$
```

02_lab_ > C 06_lab2.c

```c

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 1000000

int main() {
    double *A, *B, *C;
    double start, end;

    A = (double *)malloc(N * sizeof(double));
    B = (double *)malloc(N * sizeof(double));
    C = (double *)malloc(N * sizeof(double));

    for (int i = 0; i < N; i++) {
        A[i] = i * 1.0;
        B[i] = i * 2.0;
    }

    start = omp_get_wtime();
    for (int i = 0; i < N; i++) {
        C[i] = A[i] + B[i];
    }
    end = omp_get_wtime();
    printf("Sequential Time: %f seconds\n", end - start);

    start = omp_get_wtime();
    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        C[i] = A[i] + B[i];
    }
    end = omp_get_wtime();
    printf("Parallel Time: %f seconds\n", end - start);
}


Observations:
Here we can see that the time for the parallel execution is 0.010886 seconds
and that for sequential execution is 0.003649 seconds so we can confirm
parallel execution helps in reducing the time which is one of the main purpose of
using openmp.

```

```
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/02_lab_$ gcc -fopenmp 06_lab2.c  -o p6
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/02_lab_$ ./p6
Sequential Time: 0.003649 seconds
Parallel Time: 0.010886 seconds
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/02_lab_$
```

Ln 28, Col 2